Joshua Dalton
IT 505

Questions
When composing your answers to the following questions, it is only required that you answer in terms of TypeScript (and/or JavaScript as indicated by the question).

1. What error (aka, exception) handling control structure is available in TypeScript and what is its syntax? Describe how this structure works and what it allows us to accomplish.

TS uses “try”, “catch”, and “finally” to handle errors. The try block runs code that might cause an error. If an error happens, the catch block handles it so the program doesn't just crash. The finally block runs whether an error happens or not. Example:

```
try {
   throw new Error("Something went wrong");
} catch (error) {
   console.log("An error occurred");
} finally {
   console.log("Finished");
}
```

2. How are errors represented? What types of useful information are available about them?

They are represented as Error objects. They contain useful information such as the error's name, a message explaining what went wrong, and a stack trace showing where the error happened in the program.

3. Is it possible to explicitly generate a specific error (that is, can you use the language syntax to say “Generate an error of this type at this point in the program” without having to resort to invalid code, such as dividing by zero)? If so, how?

Yes. By using the throw keyword. You can use throw new Error() to create an error with a specific message. You can also throw specific types of errors, such as TypeError or RangeError. This is useful when we want to stop something from happening if a specific condition is met.

4. What is the nullish coalescing operator (both symbolically and in meaning) and how might it be used?

Symbolically: ??. It lets us provide a default value if a variable is null or undefined. If the variable already has a value, it uses that instead. This might be used when working with variables that might not have a value assigned to them.

5. What is the optional chaining operator (both symbolically and in meaning) and how might it be used?

Symbolically: ?. It allows you access properties or methods of an object without causing an error if the object is null or undefined. This might be useful when working with objects that have or potentially have missing information.

Joshua Dalton
IT 505

6. What happens if an error is thrown during the execution of an async function? What happens when the await operator receives a rejected promise?

The function will return a rejected promise. When await gets a rejected promise, it throws the error at that point in the code.

7. What is a module and why are modules useful? Compare and contrast the idea of a module with the idea of a class.

A module is a separate file that has code like functions, variables, and classes that can be used in other files. Modules are useful because they help organize code and make it easier to reuse. A class is used to make objects and methods, while a module is used to organize and share code between different files.

8. Deno supports two types of module: CommonJS and ES. Briefly explain the differences between these two types of modules. Which is most commonly used when writing new TS code? Why is it useful to maintain some familiarity with the other?

CommonJS = An older module system commonly used in "Node.js". It uses require() to import code and module.exports to export code.

ES = A newer, standardized module system that uses import and export to share code between files. It is the most commonly used when writing new TS code.

ES modules are preferred because they are the modern standard and work well with TS and Deno. It is still useful to understand CommonJS because many older JS programs and libraries use it.

9. How does the use of a module impact scoping of variables, classes, and functions? How is this scoping controlled in ES modules? Why is this useful?

Modules have their own scope. In ES modules, you use export to make something available to other files and import to access it. This is useful because it keeps code organized, prevents "naming conflicts", and allows you to control what parts of our code can be used in other places.

10. What is the role of an ES module's default export (aka default feature)? How many default exports may a single module have? What syntax would you use to access an ES module's default export within your program?

A default export is the main feature that a module makes available to other files. Each module can only have one default export. You use export default to create it and import to access it in another file. You can also choose your own name when importing it. Example syntax:

```
export default Class Player { //this would be in a file such as player.ts
    name = "Drake Maye";
}
```

Joshua Dalton
IT 505

import Player from "./player.ts"; //this would be in main.ts for this example

let player = new Player();
console.log(player.name);

11. What syntax would you use to access an ES module's non-default features within your program? How would you modify the local name associated with a specific feature obtained from an ES module? What shortcut can you use to access all of an ES module's available features at once, without having to list them all explicitly?

You can access non-default features using import with curly braces {} around the feature names. You can rename an imported feature using the as keyword. To import all available features at once, you can use import * as followed by a name. Example:

import { add, subtract } from "./math.ts";

import { add as addition } from "./math.ts";

import * as math from "./math.ts";

12. TypeScript provides several built-in classes, including Math, Array, and String (as we've seen previously). One that we have not yet covered is the Date class. What syntax would you use to create a new instance of the built-in Date class?

You can create a new instance of the "Date" class by using the new Date() constructor. This creates a Date object that represents the current date and time. Syntax example:

let currentDate = new Date(); //you can also put specific date by putting in parenthesis

console.log(currentDate);

13. How does a timestamp measure time? Explain how you would generate the current timestamp using the built-in Date class.

It measures time by counting the number of milliseconds that have passed since January 1, 1970. You can generate the current timestamp using Date.now(), which returns the number of milliseconds since that date.

14. How can you generate a Date instance that represents the current date and time? How can you represent specific times and/or dates using the Date class? How would you retrieve and format the value of a Date instance?

You can use new Date() to make a Date object representing the current date and time. You can provide a date string or numbers for the year, month, day, and time to represent a specific date/time. You can retrieve and format the date using methods like getFullYear(), getMonth(), toLocaleDateString(), and toLocaleTimeString().

Joshua Dalton
IT 505

15. What is Coordinated Universal Time (aka UTC – yes, the acronym doesn't match the name!)? How does it relate to Greenwich Mean Time (GMT)? How does it relate to the concept of local time? How is UTC relevant to the Date class?

UTC is the standard time used around the world. It is similar to GMT, but UTC is more precise and standardly used by computers. They are similar because they both serve as reference points for time zones around the world and have the same time offset of zero. Local time is based on UTC but adjusted depending on your time zone. In TS, the Date class stores time based on UTC.

16. The Deno Standard Library contains a datetime module. What functionality does this module provide and how does it relate to the built-in Date class?

The module provides additional tools for working with dates and times, like formatting dates, parsing date strings, and doing date calculations. It works alongside the built-in Date class by providing features that make dates easier to work with. The Date class handles basic date and time operations, while the datetime module gives more advanced functionality.

17. Deno supports several Web Platform APIs. What advantage does it gain by doing so? Probably the most commonly used of these Web Platform APIs is the fetch API. Provide a brief summary of the fetch API and its capabilities.

Deno lets you use a lot of the same features available in web browsers without the need for additional libraries. This makes it easier to write code that works in different environments. The fetch() API lets you send HTTP requests to websites and APIs to retrieve or send data. It supports methods like GET and POST and can handle responses such as JSON and text.

18. Deno supports an HTTP Server API as well. Provide a brief summary of this API and its general capabilities.

It allows you to create a web server using TS. You can use Deno.serve() to start a server that listens for requests and sends responses back to users. It can be used to create programs like websites and APIs that communicate over the internet.

19. Explain how to use the Deno Runtime API to read and write text files.

Deno reads and writes text files using Deno.readTextFile() and Deno.writeTextFile(). The read function gets the contents of a file, while the write function allows you to make a file or replace its contents. Both functions are asynchronous, so you can use await with them.

20. There are three common resources from which one might access third-party modules (that is, modules outside the Deno Standard Library) for use with Deno: deno.land/x, JSR, and NPM. Compare and contrast these three resources and explain the process and syntax one would use to access a module from it. Note: Two frequently used third-party modules that should be available via all three are Cliffy and Chalk. Since these modules may be

of use in building your final project, it is strongly recommended that you start your explorations with them.

deno.land/x = Website that hosts third party modules made for Deno. You can import modules using their URL.

JSR = "Modern package registry" designed for JS and TS. It works well with Deno and allows you to import packages using jsr:.

NPM = Large package registry originally made for Node.js. Deno also supports NPM packages, which you can import using npm:.

All three allow you to use code created by other developers instead of having to write everything yourself. The main difference is where the modules are hosted and how you import them.

21. How many third-party modules are currently available via the official Deno hosting service at https://deno.land/x? (Hint: The page displays the total number in the empty search field.) Look through some of the options available. Identify one that you think might be useful and report its name and number of stars and summarize what it does.

There are 7741 modules. I chose Cliffy, it has 1.2k stars. It lets you make command-line programs with menus, prompts, and commands. I think it would be useful for making my TS projects easier to use, especially programs that need user input

22. Describe some ways that the clean code philosophy could be meaningfully applied to the material you've discovered in this unit.

Clean code can be applied to this unit by keeping programs organized and making them easier to read and understand. I can use modules to separate different parts of a program, give variables and functions clear names, and use error handling to prevent unexpected problems. I can also avoid repeating code by importing functions from other modules, making programs easier to maintain, debug, and update in the future.